CAMPUS PULSE • TECHNICAL SPECIFICATION

Complete Project Understanding & System Knowledge Base

A definitive, evidence-based architectural compendium, source-code analysis, database specification, API catalog, and operational manual.

Application:	Campus Pulse (Multi-Role Campus Operations System)
Repository:	`CampusPulse_co`
Primary Stack:	FastAPI 0.109, Python 3.14, SQLite 3 (WAL), React 19, Vite, Tailwind
Investigation Date:	October 2026
Verification Scope:	100% Source-Code Backed • Zero Speculative Additions
Document Suite:	7 Comprehensive Technical Volumes

Knowledge Suite Table of Contents

Structured Volume Navigation

1. **Part I: Master Technical Architecture & System Context** (Architecture, 20 Layers, Roles, Security, Background Jobs)
2. **Part II: Project Knowledge Index & Navigation Guide** (Repository Map, Inventory, Confidence Ratings, Coverage Checklist)
3. **Part III: End-to-End Operational Workflows** (Complaint Lifecycles, Multi-Location Routing, 3-Tier SLA Escalation)
4. **Part IV: Relational Database & Schema Specification** (SQLite WAL Configuration, 35 Active Tables, Constraint Topology)
5. **Part V: Complete REST API Endpoint Reference** (277 Discovered Endpoints across 24 Routers with Authentication Scopes)
6. **Part VI: Technical Issues, Architectural Gaps & Risk Register** (10 Tracked Technical Issues, Bottlenecks, Remediations)
7. **Part VII: AI Handoff Prompt & Condensed Context** (Curated System Truths, Operational Constraints for AI Sessions)

CAMPUS PULSE KNOWLEDGE SUITE

Part I: Master Technical Architecture & System Context

Source: `CAMPUS_PULSE_MASTER_CONTEXT.md`

Campus Pulse — Master Technical Architecture and System Context

Document ID: `CP-DOC-001`
Classification: Definitive Architecture and System Reference
Status: Complete & Verified Against Source Code
Date: October 2026
Repository: `CampusPulse_co`

Executive Summary

Campus Pulse is an enterprise-grade campus operations and incident resolution management platform engineered for higher education institutions. The system bridges communication gaps between students, facility wardens, departmental coordinators, facility managers, and university administrators.

The platform provides structured, auditable grievance tracking, multi-tier Service Level Agreement (SLA) escalation, facility-based access control, real-time WebSocket notifications, and an administrative oversight dashboard.

This document serves as the **definitive technical specification and master system context**. Every claim, architectural layer, database schema definition, security policy, and workflow described herein has been directly verified against the repository's source code.

---

1. Product Identity and Core Capabilities

1.1 Problem Space

Traditional university campuses manage maintenance, infrastructure failures, mess grievances, hostel issues, and academic complaints through ad-hoc mechanisms (paper logbooks, unmonitored WhatsApp groups, verbal complaints to wardens, or unstructured email threads). This results in: - Unassigned or forgotten complaints. - Zero accountability for resolution turnaround times. - Lack of visibility for students into ticket status. - Inability for campus leadership to identify recurring infrastructure bottlenecks or vendor failures.

1.2 Campus Pulse Solution

Campus Pulse digitizes and enforces the entire campus resolution lifecycle: - **Strict Role-Based Multi-Tenancy:** 6 clearly separated roles with object-level scoping. - **Location & Department Routing:** Two-factor ticket routing linking physical facilities (hostels, academic blocks, mess halls) to operational departments (electrical, plumbing, housekeeping, IT). - **Enforced SLAs and Escalation Trees:** Automatic time-based escalation from coordinators to App Managers and Campus Administrators when deadlines expire. - **Stateful Auditable Lifecycles:** Cryptographic session management, comprehensive audit logging of all state transitions, and 48-hour student verification windows before closure.

---

2. Comprehensive 20-Layer Architectural Blueprint

Campus Pulse is structured across 20 distinct architectural layers spanning the backend, frontend, database, and infrastructure tiers:

```
graph TD
    subgraph Frontend Tier
        L1[Layer 1: UI Presentation React 19 & Tailwind]
        L2[Layer 2: Component & Layout Engine Lucide React]
        L3[Layer 3: Client State Management React Hooks & Context]
        L4[Layer 4: Client API Client Axios with Interceptors]
        L5[Layer 5: Client-Side Routing React Router v6]
    end

    subgraph API & Gateway Tier
        L6[Layer 6: ASGI Web Framework FastAPI 0.109]
        L7[Layer 7: Modular API Routing 24 Route Submodules]
        L8[Layer 8: Input Validation & Serialization Pydantic v2]
        L9[Layer 9: Real-time Communication WebSockets]
    end

    subgraph Security & Middleware Tier
        L10[Layer 10: Security Middleware CORS, Headers, Rate-Limits]
        L11[Layer 11: Stateful Authentication SHA-256 Bearer Sessions]
        L12[Layer 12: Role & Object-Level Access Control RBAC]
    end

    subgraph Core Business Tier
        L13[Layer 13: Complaint & Routing Services]
        L14[Layer 14: Notification & Dispatch Engine Email & Push]
        L15[Layer 15: Background Scheduler 300s SLA Worker Thread]
        L16[Layer 16: Audit Logging & Event Tracking]
    end

    subgraph Persistence & Infrastructure Tier
        L17[Layer 17: Database Abstraction SQLAlchemy ORM]
        L18[Layer 18: Relational Storage SQLite 3 with PRAGMA WAL]
        L19[Layer 19: Local Asset Storage Uploads Directory]
        L20[Layer 20: Build & Runtime Toolchain Vite & Uvicorn]
    end

    L1 --> L2 --> L3 --> L4 --> L5
    L4 -->|HTTP/REST| L6
    L3 -->|WebSocket| L9
    L6 --> L7 --> L8
    L7 --> L10 --> L11 --> L12
    L12 --> L13 --> L14
    L13 --> L16
    L15 --> L13
    L13 --> L17 --> L18
    L13 --> L19
```

Detailed Layer Specifications

1. **Presentation Layer (`Frontend/src/pages/`):** Responsive dashboard pages rendered via React 19, providing tailored UI workflows for Students, Coordinators, Wardens, App Managers, and Administrators.
2. **Component & Layout Layer (`Frontend/src/components/`):** Reusable design system utilizing Tailwind CSS utility classes and Lucide React iconography.
3. **Client State Layer (`Frontend/src/context/`):** React Context and custom hooks (`AuthContext` , notification listeners) providing synchronized authentication and active session states.
4. **Client Network Layer (`Frontend/src/api/client.js`):** Axios instance configured with request interceptors for bearer token injection and response interceptors for global 401 session expiration handling.
5. **Client Routing Layer (`Frontend/src/App.jsx`):** React Router DOM v6 handling declarative path routing, protected route guards, and role-based redirects.
6. **ASGI Web Framework (`Backend/main.py`):** High-performance asynchronous FastAPI 0.109 backend running on Python 3.14.2 and Uvicorn.
7. **API Routing Layer (`Backend/api/routes/`):** 24 distinct APIRouters organizing 277 REST endpoints into modular functional domains.
8. **Validation & Serialization Layer (`Backend/schemas/`):** Pydantic v2 models validating incoming JSON payloads and structuring outgoing serialization models.
9. **Real-Time Communication Layer (`Backend/api/routes/websockets.py`):** WebSocket server maintaining active duplex communication channels for instant notification pushes.
10. **Security Middleware Layer (`Backend/main.py`):** Global CORS middleware handling origin verification, preflight checks, and standard security headers.
11. **Stateful Authentication Layer (`Backend/core/security.py` , `Backend/api/routes/auth.py`):** Database-backed session mechanism using SHA-256 hashed bearer tokens (`secrets.token_urlsafe(32)`) and PBKDF2-HMAC-SHA256 password hashing.
12. **Role & Object Access Control (`Backend/api/deps.py` , `Backend/api/routes/complaints.py`):** Declarative FastAPI dependency injection enforcing both user role boundaries and spatial object scoping (`_visible_complaints`).
13. **Complaint & Routing Business Services (`Backend/services/complaint_service.py`):** Core domain logic managing complaint dispatching, status transitions, coordinator assignment, and reopen limits.
14. **Notification Dispatch Engine (`Backend/services/notification_service.py`):** Multi-channel alert dispatcher publishing in-app notifications, WebSocket events, and SMTP email alerts.
15. **Background Scheduler (`Backend/services/scheduler_service.py`):** In-process background worker executing every 300 seconds to evaluate overdue SLAs and auto-close verified complaints.
16. **Audit & Event Tracking Layer (`Backend/models/audit.py`):** Immutable transaction ledger recording every status change, assignment update, and administrative intervention.
17. **ORM Database Abstraction (`Backend/database/session.py` , `Backend/models/`):** SQLAlchemy ORM models abstracting database schema and query construction.

18. **Relational Storage Layer (`Backend/storage/campus_pulse.db`):** SQLite 3 configured with Write-Ahead Logging (`PRAGMA journal_mode=WAL`) and a 5,000ms busy timeout.
19. **File Storage Layer (`Backend/storage/uploads/`):** Local filesystem storage saving uploaded complaint evidence files with UUID-based filenames.
20. **Build & Runtime Toolchain:** Node.js Vite build system for frontend bundling and Python Uvicorn ASGI server for backend process execution.

---

3. User Roles and Permission Matrix

Campus Pulse implements 6 distinct user roles. Access control is strictly enforced at both the endpoint level (HTTP 403 Forbidden) and the object level (SQL row filtering).

3.1 Role Hierarchy and Scope

Role Identifier	Display Name	Operational Purpose	Spatial Scope
`super_admin`	Super Administrator	Multi-tenant setup, global system configuration, audit review	System-wide (Blocked from operational complaints)
`administrator`	Administrator	Overall campus operations, user approvals, master escalation target	Institution-wide
`app_manager`	Application Manager	Facility operations overseer, first-tier escalation receiver	Assigned Locations
`location_in_charge`	Warden / In-Charge	Residential facility supervisor, physical verification	Assigned Facility / Hostel
`coordinator`	Department Coordinator	Field resolver (Electrical, Plumbing, IT, Mess)	Assigned Department + Location
`student`	Student	End-user submitting grievances, tracking resolution	Personal + Public in own location

3.2 Granular Permissions Matrix

Feature / Action	Student	Coordinator	Warden	App Manager	Admin	Super Admin
Submit Complaint	✅	❌	❌	❌	❌	❌
View Complaint List	Own + Public	Dept + Loc	Assigned Loc	Assigned Loc	All	❌ (403)
Update Status (In Progress/Resolved)	❌	✅	✅	✅	✅	❌
Verify & Close Complaint	✅ (Own)	❌	❌	❌	✅	❌
Reopen Complaint (Max 3x)	✅ (Own)	❌	❌	❌	❌	❌
Provide Feedback & Rating	✅ (Own)	❌	❌	❌	❌	❌
Publish Campus Announcement	❌ (403)	❌ (403)	✅ (Loc)	✅ (Loc)	✅	✅
Create Users & Assign Roles	❌	❌	❌	❌	✅	✅
Assign Facilities to Wardens/Managers	❌	❌	❌	❌	✅	✅
View System Audit Logs	❌	❌	❌	❌	✅	✅
Configure System Settings	❌	❌	❌	❌	✅	✅

---

4. End-to-End Operational Workflows

4.1 Complaint Lifecycle and Multi-Tier SLA Escalation

```
stateDiagram-v2
    [*] --> Submitted: Student files complaint
    Submitted --> Assigned: Auto-routed or Assigned to Coordinator
    Assigned --> In_Progress: Coordinator accepts & begins work
    In_Progress --> Resolved: Coordinator marks work complete
    Resolved --> Waiting_for_Verification: 48-Hour Student Review Window

    Waiting_for_Verification --> Closed: Student verifies resolution
    Waiting_for_Verification --> Closed_Auto: 48 Hours expire with no action
    Waiting_for_Verification --> Reopened: Student disputes resolution (Max 3x)

    Reopened --> In_Progress: Coordinator re-investigates
    Closed --> [*]
    Closed_Auto --> [*]
```

SLA Escalation Mechanics:

1. **Tier 1 (Overdue):** If a complaint remains unresolved beyond `estimated_resolution_time` (default 48h), status shifts to Overdue, and the assigned Coordinator receives an immediate alert.
2. **Tier 2 (Overdue + 2 Days):** If still unresolved 48 hours after Tier 1, system dispatches an escalation notification to the responsible **App Manager** mapped to that location.
3. **Tier 3 (Overdue + 4 Days):** If still unresolved 96 hours after Tier 1, complaint escalates directly to the campus **Administrator** dashboard with a critical urgency badge.

4.2 Account Provisioning and Activation Workflow

1. **Creation:** Administrator creates a new account via `POST /api/users`. Account is initialized with `is_active=False`.
2. **Token Generation:** System generates a 32-byte cryptographic random URL-safe token stored in `activation_tokens` with a 24-hour expiration timestamp.
3. **Email Dispatch:** Activation email containing link `https://<domain>/activate?token=<hex>` is sent to the user's institutional email address.
4. **Activation:** User clicks the link, inputs their new password; backend verifies token hash, sets PBKDF2 hashed password, sets `is_active=True`, marks token used, and establishes the initial active session.

---

5. Database Architecture & Schema Specification

5.1 Storage Engine Configuration

- **Database File:** `Backend/storage/campus_pulse.db`
- **Engine:** SQLite 3
- **Journal Mode:** `PRAGMA journal_mode = WAL;` (Write-Ahead Logging enabled for concurrent multi-reader access)
- **Locking Timeout:** `PRAGMA busy_timeout = 5000;` (5-second wait queue on write lock contention)
- **Foreign Keys:** `PRAGMA foreign_keys = ON;` (Referential integrity enforced)

5.2 Active Core Schema Catalog (35 Active Tables)

```
erDiagram
    USERS ||--o{ USER_SESSIONS : maintains
    USERS ||--o{ COMPLAINTS : submits
    USERS ||--o{ COMPLAINT_ASSIGNMENTS : assigned_to
    USERS ||--o{ COMPLAINT_AUDIT : performs
    USERS ||--o{ NOTIFICATIONS : receives
    LOCATIONS ||--o{ COMPLAINTS : located_at
    DEPARTMENTS ||--o{ COMPLAINTS : handled_by
    DEPARTMENTS ||--o{ COORDINATOR_ASSIGNMENTS : staffs
    LOCATIONS ||--o{ COORDINATOR_ASSIGNMENTS : located_in
    COMPLAINTS ||--o{ COMPLAINT_ATTACHMENTS : includes
    COMPLAINTS ||--o{ COMPLAINT_FEEDBACK : receives
    COMPLAINTS ||--o{ COMPLAINT_AUDIT : tracks
    USERS ||--o{ ANNOUNCEMENTS : publishes
```

Core Operational Tables:

1. `users`: Core identity credentials, PBKDF2 password hashes, role strings, department/location bindings, and active flags.
2. `user_sessions`: Active bearer tokens, SHA-256 token hashes, creation dates, last activity timestamps, and client IP/user-agent metadata.
3. `activation_tokens`: Ephemeral cryptographic tokens used for account activation and password reset sequences.
4. `locations`: Campus physical assets (hostels, academic blocks, mess halls, sports complexes, libraries).
5. `departments`: Operational departments (electrical, plumbing, housekeeping, civil, network, administrative).
6. `complaints`: Central grievance entity containing ticket title, description, priority, category, status, creator FK, location FK, department FK, SLA target dates, and reopen counters.
7. `complaint_assignments`: Historical and current mapping between complaints and assigned staff users.
8. `complaint_attachments`: File metadata (original filename, storage disk path, MIME type, file size, uploader FK).
9. `complaint_feedback`: Student ratings (1 to 5 stars), qualitative feedback text, and resolution satisfaction flags.
10. `complaint_audit`: Tamper-evident transaction log recording action, old status, new status, actor user ID, and timestamp.
11. `announcements`: Official broadcast notices with title, content, target audience scope, location filter, and author FK.
12. `notifications`: In-app notification queue records with read/unread status, notification type, and payload links.
13. `coordinator_assignments`: Many-to-many relationship mapping coordinators to specific physical locations and departments.
14. `app_manager_assignments`: Mapping responsible App Managers to designated campus facilities.
15. `location_in_charge_assignments`: Mapping Wardens to specific residential halls or facilities.

(Note: The database contains ~160 additional enterprise placeholder tables created from legacy raw SQL schema migrations which are currently dormant and unmapped in active code.)

---

6. Security and Authentication Architecture

6.1 Password Security

- **Algorithm:** PBKDF2 with HMAC-SHA256 (`hashlib.pbkdf2_hmac`).
- **Iterations:** 100,000 rounds.
- **Salt:** 16 bytes of cryptographically secure random bytes generated via `secrets.token_bytes(16)`.
- **Storage Format:** Salt and hash stored as hexadecimal strings.

6.2 Session Management

- **Token Format:** 32-byte URL-safe string generated via `secrets.token_urlsafe(32)`.
- **Database Storage:** Tokens are hashed with SHA-256 before being persisted in `user_sessions`. Plaintext bearer tokens are never stored at rest.

- **Session Validation:** Incoming requests pass `Authorization: Bearer <token>`. Dependency `get_current_user` computes `hashlib.sha256(token.encode()).hexdigest()`, looks up the session record, validates that `expires_at > now()`, and loads the associated user.
- **Revocation:** Logout operations execute `DELETE FROM user_sessions WHERE token_hash = :hash`, instantly invalidating the session.

6.3 Input Validation and Sanitization

- All incoming payloads pass through Pydantic v2 validation models enforcing length constraints, regex validation, email formats, and type coercion.
- SQL injection is eliminated through SQLAlchemy's parameterized query engine.
- XSS prevention is provided by React's automatic JSX text escaping and strict MIME-type validation on file uploads.

---

7. Frontend Architecture and Client State

7.1 Technology Stack

- **Framework:** React 19 (`Frontend/package.json`)
- **Build Tool:** Vite 5 with Hot Module Replacement (HMR)
- **Styling:** Tailwind CSS 3 with PostCSS and Autoprefixer
- **Icons:** Lucide React
- **HTTP Client:** Axios with centralized interceptors
- **Routing:** React Router DOM v6

7.2 Directory Organization

- `Frontend/src/api/` : Centralized HTTP request modules (`client.js` , `auth.js` , `complaints.js` , `users.js` , `announcements.js`).
- `Frontend/src/components/` : Modular UI blocks (modals, status badges, navbar, sidebar, data tables, file upload dropzones).
- `Frontend/src/context/` : Global state providers (`AuthContext.jsx` , `NotificationContext.jsx`).
- `Frontend/src/pages/` : Role-specific view controllers:
- `StudentDashboard.jsx` : Complaint submission, personal ticket tracking, feedback modal.
- `CoordinatorDashboard.jsx` : Department queue, status update controls, proof upload.
- `WardenDashboard.jsx` : Facility overview, hostel grievance queue.
- `AppManagerDashboard.jsx` : Multi-facility metrics, SLA escalation queue.
- `AdminDashboard.jsx` : Campus analytics, user management, location & department management.

---

8. Background Jobs and Scheduled Services

Campus Pulse incorporates an automated background processing engine implemented in `Backend/services/scheduler_service.py` :

- **Execution Model:** In-process daemon thread started during FastAPI lifespan initialization.
- **Execution Frequency:** Every 300 seconds (5 minutes).
- **Scheduled Tasks:**
- **48-Hour Auto-Closure:**
  - Query: `SELECT * FROM complaints WHERE status = 'Resolved' AND resolved_at <= :threshold_48h`
  - Action: Updates status to `Closed` , sets resolution note to `"Auto-closed after 48h without student verification"` , inserts audit record, and pushes notification to the student.
- **Tier-1 SLA Check (Overdue):**
  - Query: `SELECT * FROM complaints WHERE status IN ('Submitted', 'Assigned', 'In Progress') AND estimated_resolution_time < :now AND is_overdue = 0`
  - Action: Sets `is_overdue = 1` , logs audit event, and alerts assigned Coordinator.
- **Tier-2 SLA Escalation (+2 Days):**
  - Query: `SELECT * FROM complaints WHERE is_overdue = 1 AND estimated_resolution_time <= :threshold_now_minus_2d AND escalation_level = 0`
  - Action: Sets `escalation_level = 1` , alerts responsible App Manager for the location.
- **Tier-3 SLA Escalation (+4 Days):**
  - Query: `SELECT * FROM complaints WHERE is_overdue = 1 AND estimated_resolution_time <= :threshold_now_minus_4d AND escalation_level = 1`
  - Action: Sets `escalation_level = 2` , alerts Campus Administrator.

---

9. Configuration, Ports, and Environment

9.1 Network Port Allocation

- **Backend API Server:** Port `8000` (`http://localhost:8000`)
- **Frontend Development Server:** Port `5173` (`http://localhost:5173`)
- **API Documentation (Swagger UI):** `http://localhost:8000/docs`
- **Alternative Docs (ReDoc):** `http://localhost:8000/redoc`

9.2 Key Environment Variables (`Backend/.env`)

Variable	Default Value	Purpose
`DATABASE_URL`	`sqlite:///./storage/campus_pulse.db`	SQLAlchemy connection string
`SECRET_KEY`	*(Random 64-char string)*	Cryptographic signing secret
`SMTP_HOST`	`smtp.gmail.com`	Mail transfer agent server host
`SMTP_PORT`	`587`	TLS SMTP communication port
`SMTP_USER`	*(Configured email)*	Institutional outbound sender account
`SMTP_PASSWORD`	*(App Password)*	Outbound SMTP credentials
`ENVIRONMENT`	`development` / `production`	Application operational mode

---

10. Source-of-Truth Code Matrix

Feature / Domain	Backend Implementation File	Frontend Implementation File
Authentication & Sessions	`Backend/api/routes/auth.py` , `Backend/core/security.py`	`Frontend/src/context/AuthContext.jsx` , `Frontend/src/pages/Login.jsx`
Complaint Management	`Backend/api/routes/complaints.py` , `Backend/services/complaint_service.py`	`Frontend/src/pages/StudentDashboard.jsx` , `Frontend/src/pages/ComplaintsList.jsx`
Location & Department Routing	`Backend/api/routes/locations.py` , `Backend/api/routes/departments.py`	`Frontend/src/pages/AdminDashboard.jsx`
Real-time Notifications	`Backend/api/routes/websockets.py` , `Backend/services/notification_service.py`	`Frontend/src/components/NotificationBell.jsx`
Announcements	`Backend/api/routes/announcements.py`	`Frontend/src/pages/Announcements.jsx`
Background Escalation	`Backend/services/scheduler_service.py`	N/A (Server background thread)
Audit Logging	`Backend/models/audit.py` , `Backend/services/audit_service.py`	`Frontend/src/pages/AuditLogs.jsx`
Feedback & Ratings	`Backend/api/routes/feedback.py` , `Backend/models/feedback.py`	`Frontend/src/components/FeedbackModal.jsx`

CAMPUS PULSE KNOWLEDGE SUITE

Part II: Project Knowledge Index & Navigation

Source: `CAMPUS_PULSE_KNOWLEDGE_INDEX.md`

Campus Pulse — Project Knowledge Index & Master Navigation Guide

Document Type: Master Directory & Evidence Index
Repository: Campus Pulse (`CampusPulse_co`)
Status: Production-Verified against Active Codebase
Audience: AI Coding Assistants, Lead Architects, Product Reviewers, New Engineers

---

1. Executive Directory Overview

This documentation suite represents an exhaustive, source-code-driven knowledge extraction of **Campus Pulse**, an enterprise campus operations and complaint lifecycle management platform. Every claim, schema definition, route mapping, and business rule within these files is backed by direct evidence from the repository files, SQLite system catalogs, or automated test executions.

All deliverables requested in the knowledge extraction specification have been generated in this dedicated directory:

Document Deliverable	Filename	Purpose & Target Content	Primary Evidence Sources
Deliverable A: Master Context	`CAMPUS_PULSE_MASTER_CONTEXT.md`	The definitive technical reference covering architecture, all 6 roles, permissions, 20 layers, data model, security, frontend, jobs, and deployment.	Codebase root, `Backend/api/` , `Backend/services/` , `Frontend/src/`
Deliverable B: API Reference	`CAMPUS_PULSE_API_REFERENCE.md`	Structured catalog of all 277 registered REST endpoints across 24 domain routers with HTTP methods, scopes, schemas, and test coverage.	`Backend/api/main.py` , `Backend/api/routers/*.py`
Deliverable C: Database Reference	`CAMPUS_PULSE_DATABASE_REFERENCE.md`	Complete SQLite relational model (196 tables), foreign keys, indexes, PRAGMA WAL mode, constraints, and active vs. placeholder schema analysis.	`Backend/storage/campus_pulse.db` , SQLite `PRAGMA` catalogs
Deliverable D: Workflows Reference	`CAMPUS_PULSE_WORKFLOWS.md`	Mermaid state and sequence diagrams for the complete complaint lifecycle, multi-location assignment, escalation, verification, and notices.	`complaint_service.py` , `auth_service.py` , `test_workflow*.py`
Deliverable E: Issues & Gaps Register	`CAMPUS_PULSE_ISSUES_AND_GAPS.md`	Consolidated defect and risk registry with unique IDs, severity, evidence, reproduction paths, and documentation contradictions.	Bug analysis, concurrency limits, schema audit
Deliverable F: Knowledge Index	`CAMPUS_PULSE_KNOWLEDGE_INDEX.md`	*(This Document)* Master navigation guide, evidence confidence labeling, and coverage checklist.	Knowledge Base Meta
Deliverable G: Context for ChatGPT	`CAMPUS_PULSE_CONTEXT_FOR_CHATGPT.md`	Compact, high-signal handoff prompt designed to prime external AI models with core facts, rules, and constraints without hallucinations.	Distilled Architectural Summary

---

2. Core Architectural Facts (The "Must-Knows")

Any developer or AI assistant continuing work on Campus Pulse must understand these foundational facts:

1. **Active Technology Stack:**
2. **Backend:** Python 3 (CPython 3.10-3.14, verified on Python 3.14.2) using **FastAPI** (`0.109.0+`) running on **Uvicorn** (`0.27.0+`).
3. **Frontend: React 19** (`19.2.7`) with **Vite** (`8.1.3`), **React Router DOM v7** (`7.18.1`), and **Tailwind CSS v4** (`4.3.2`).
4. **Database: SQLite 3** operating in **WAL (Write-Ahead Logging)** mode (`Backend/storage/campus_pulse.db`).
5. **Stateful Session Architecture:**
6. The application does **not** use stateless JWTs.
7. It issues high-entropy opaque bearer tokens (`secrets.token_urlsafe(32)`), hashes them with SHA-256, and validates them against the `user_sessions` SQLite table.
8. This enables **immediate session invalidation** upon password changes or administrative deactivation.
9. **No SMS OTPs:**
10. The application does **not** use 6-digit numeric SMS OTPs.
11. It utilizes 32-byte cryptographic **Activation Tokens** (`activation_tokens`) delivered via email for account initialization and password resets.
12. **The Six User Roles:**
13. `super_admin` : Platform-level multi-tenant administration (strictly blocked from campus operational data).

14. `administrator`: Campus-level authority (user management, facilities, backups, settings).
15. `app_manager`: Multi-location operations supervisor (reassigns complaints across assigned hostels/complexes).
16. `location_in_charge`: Facility-specific warden (manages facility complaints, posts hostel circulars).
17. `coordinator`: Departmental technician/head (resolves complaints, uploads proof; **strictly forbidden from posting announcements**).
18. `student`: Resident student (submits issues, upvotes public notices, verifies or reopens resolutions; **strictly forbidden from posting announcements**).
19. **Zero-Drop Routing & 48-Hour Student Verification:**
20. Complaints follow an automated routing matrix: Coordinator Assignment → Facility Warden → App Manager → Administrator.
21. Staff cannot arbitrarily close issues. When marked resolved, issues enter `'Waiting for Verification'` for 48 hours. If the student takes no action, the background scheduler auto-closes them. If the student rejects within 48 hours, the issue is reopened (up to 3 times before administrative escalation).

---

3. Evidence Coverage Checklist

The following repository directories, modules, and systems were thoroughly inspected for this audit:

- [x] **Backend Entry Points:** `Backend/run_api.py`, `Backend/api/main.py`.
- [x] **API Routers (24 modules):** `auth.py`, `users.py`, `departments.py`, `locations.py`, `issues.py`, `complaints.py`, `public_complaints.py`, `dashboard.py`, `notifications.py`, `settings.py`, `attachments.py`, `analytics.py`, `audit.py`, `lost_found.py`, `announcements.py`, `feedback.py`, `reports.py`, `search.py`, `backups.py`, `chat.py`, `supervisor.py`, `master.py`, `programs.py`, `platform.py`.
- [x] **Core Business Services (26 services):** `auth_service.py`, `complaint_service.py`, `notification_service.py`, `announcement_service.py`, `attachment_service.py`, `audit_service.py`, `backup_service.py`, `scheduler_service.py`, `location_service.py`, `department_service.py`, etc.
- [x] **Domain Models (9 models):** `user.py`, `student.py`, `coordinator.py`, `location_in_charge.py`, `app_manager.py`, `administrator.py`, `location.py`, `department.py`, `complaint.py`.
- [x] **Security & Middleware:** `Backend/api/security.py`, `Backend/middleware/rate_limiter.py`.
- [x] **Database File & Schema:** `Backend/storage/campus_pulse.db` (196 tables cataloged via `PRAGMA table_info`, `foreign_key_list`, `index_list`).
- [x] **Frontend Shell & Config:** `Frontend/index.html`, `Frontend/package.json`, `Frontend/vite.config.js`, `Frontend/src/main.jsx`, `Frontend/src/App.jsx`.
- [x] **Frontend Routing & Guards:** `Frontend/src/routes/AppRoutes.jsx`, `Frontend/src/routes/ProtectedRoute.jsx`.
- [x] **Frontend UI Pages (40 pages):** Auth pages, Dashboards (Student, Admin, Manager, Coordinator, Location, SuperAdmin), Complaints, Announcements, Lost & Found, Notifications, Settings, Backups, Users.
- [x] **Frontend Component Library (18 components):** `Button`, `Modal`, `Table`, `Toast`, `Badge`, `Card`, `StatCard`, etc.
- [x] **Test Infrastructure:** 31 test files in `Backend/tests/` (executed smoke tests, workflow 8, workflow 4, role location assignments) and Vitest in `Frontend/src/__tests__/`.

CAMPUS PULSE KNOWLEDGE SUITE

Part III: End-to-End Operational Workflows

Source: `CAMPUS_PULSE_WORKFLOWS.md`

Campus Pulse — End-to-End Workflows, State Machines & Execution Sequences

Document Type: Core Workflows Specification & Sequence Diagrams
Repository: Campus Pulse (`CampusPulse_co`)
Status: Ground-Truth Verified against Source Code

---

1. Complaint Lifecycle State Machine

The complaint state machine defines all permitted states, actors, and transition preconditions implemented in `Backend/services/complaint_service.py` and `Backend/models/complaint.py` .

```
stateDiagram-v2
    [*] --> Submitted : Student / Warden Creates
    Submitted --> Assigned : Routing Engine / Admin Assigns
    Assigned --> In_Progress : Coordinator Takes Responsibility
    In_Progress --> Resolved : Coordinator Completes (Proof Required)
    Resolved --> Waiting_for_Verification : System Starts 48h Window
    Waiting_for_Verification --> Closed : Student Approves
    Waiting_for_Verification --> Closed_Auto : 48h Timer Expires (Scheduler)
    Waiting_for_Verification --> Reopened : Student Rejects (Max 3x)
    Reopened --> In_Progress : Coordinator Reassesses
    Submitted --> Rejected : Admin / Warden Rejects Invalid Issue
    Assigned --> Rejected : Coordinator Rejects Invalid Issue
    Closed --> [*]
    Closed_Auto --> [*]
    Rejected --> [*]
```

Step-by-Step Transition Analysis

From State	To State	Initiating Actor	Endpoint / Function	Preconditions & Validation Rules	Database Mutations	Notifications Dispatched
`[*]`	`Submitted`	Student, Warden	`POST /complaints`	Valid location & dept; title min 5 chars; desc min 10 chars; hostel residency match	`complaints` INSERT; `attachments` INSERT	Student confirmation; location resident alert (if public)
`Submitted`	`Assigned`	System / Admin	`assign_work()`	Coordinator active in department/location assignment table	`complaints.assigned_to` , `complaints.status = 'Assigned'`	Assigned Coordinator alerted
`Assigned`	`In Progress`	Coordinator	`PUT /complaints/:id/status`	Current status is `Assigned` ; caller is assigned staff or admin	`complaints.status = 'In Progress'` , `updated_at`	Complainant Student alerted
`In Progress`	`Resolved / Waiting for Verification`	Coordinator, Staff	`PUT /complaints/:id/resolve`	Current status is `In Progress` ; must provide "After Repair" image OR resolution notes	`complaints.status = 'Waiting for Verification'` , `resolved_at` , `verification_due_at`	Student alerted with verification action
`Waiting for Verification`	`Closed`	Student Author	`PUT /complaints/:id/verify`	Current status is `Waiting for Verification` ; caller is complaint creator	`complaints.status = 'Closed'` , `closed_at` , `verification_reports`	Coordinator & Student closure confirmation
`Waiting for Verification`	`Closed (Auto Closed)`	Background Daemon	`apply_automatic_rules()`	`datetime.utcnow() > verification_due_at` (48 hours elapsed)	`complaints.status = 'Closed (Auto Closed)'` , `closed_at`	System event logged; student and staff notified
`Waiting for Verification`	`Reopened`	Student Author	`POST /complaints/:id/reopen`	Current status is `Waiting for Verification` ; `reopen_count < max_reopen_count` (3)	`complaints.status = 'Reopened'` , `reopen_count += 1`	Coordinator alerted; if `reopen_count >= 2` , Admin alerted
`Submitted / Assigned`	`Rejected`	Admin, Warden, Coord	`PUT /complaints/:id/status`	Valid rejection reason entered; caller has authority	`complaints.status = 'Rejected'` , `rejection_reason`	Student alerted with rejection reason

---

2. Multi-Location & Coordinator Routing Sequence

This workflow traces how a newly submitted complaint is routed to the appropriate department coordinator across decentralized campus locations.

```
sequenceDiagram
    autonumber
    actor Student as Student
    participant Router as API Router (complaints.py)
    participant Svc as ComplaintService
    participant Auth as AuthService
    participant DB as SQLite (campus_pulse.db)
    participant Notif as NotificationService
    actor Staff as Coordinator / Warden

    Student->>Router: POST /complaints (location_id, department_id, description)
    Router->>Svc: create_complaint(user, location_id, department_id, ...)
    Svc->>Auth: get_location(location_id) & get_department(department_id)
    Auth-->>Svc: Confirms both active

    rect rgb(240, 248, 255)
        Note over Svc,Auth: Routing Engine Evaluation
        Svc->>Auth: list_coordinator_assignments(location_id, department_id)
        alt Coordinator Assigned for Location & Department
            Auth-->>Svc: Returns Coordinator ID (e.g., USR_COORDINATOR)
            Svc->>Svc: Set assigned_to = Coordinator ID, status = 'Assigned'
        else Fallback to Facility Warden
            Svc->>Auth: Check locations.warden_id
            alt Facility Warden Assigned
                Auth-->>Svc: Returns Warden ID (e.g., USR_WARDEN)
                Svc->>Svc: Set assigned_to = Warden ID, status = 'Assigned'
            else Fallback to Unassigned
                Svc->>Svc: Set assigned_to = NULL, status = 'Submitted'
            end
        end
    end

    Svc->>DB: INSERT INTO complaints (...)
    DB-->>Svc: Success
    Svc->>Notif: notify(student_id, "Complaint Submitted")
    opt If Assigned
        Svc->>Notif: assigned(complaint_id, assigned_to)
        Notif->>Staff: In-App / WebSocket Notification
    end
    Svc-->>Router: Returns Complaint Object
    Router-->>Student: 201 Created (JSON Response)
```

---

3. SLA Escalation & Background Daemon Workflow

This workflow traces the 300-second periodic scheduler executed by `Backend/services/scheduler_service.py`.

```
sequenceDiagram
    autonumber
    participant Daemon as Background Scheduler Thread (300s)
    participant CompSvc as ComplaintService
    participant DB as SQLite Storage
    participant Notif as NotificationService
    actor Admin as App Manager / Admin

    loop Every 300 Seconds
        Daemon->>CompSvc: apply_automatic_rules()
        CompSvc->>DB: SELECT * FROM complaints WHERE status NOT IN ('Closed', 'Rejected')
        DB-->>CompSvc: Returns Active Complaints

        rect rgb(255, 245, 245)
            Note over CompSvc: SLA Breach Evaluation
            loop For Each Active Complaint
                alt Overdue: current_time > sla_deadline
                    CompSvc->>DB: UPDATE complaints SET is_escalated = 1
                    alt Overdue >= 2 Days
                        CompSvc->>Notif: Alert App Manager (Overdue 2+ Days)
                        Notif->>Admin: Priority In-App Alert
                    end
                    alt Overdue >= 4 Days
                        CompSvc->>Notif: Alert Campus Administrator (Critical Delay)
                        Notif->>Admin: Critical Dashboard Banner
                    end
                end
                alt Waiting for Verification Expired (> 48h)
                    CompSvc->>DB: UPDATE complaints SET status = 'Closed (Auto Closed)'
                    CompSvc->>Notif: Notify Student & Staff of Auto-Closure
                end
            end
        end
    end
```

---

4. User Account Lifecycle & Session Security Workflow

This workflow traces how user accounts are created, activated via cryptographic tokens, authenticated via PBKDF2, and securely deactivated.

```
sequenceDiagram
    autonumber
    actor Admin as Administrator
    participant API as FastAPI Backend
    participant AuthSvc as AuthService
    participant Mail as SMTP Transport (email.py)
    actor Student as Student
    participant DB as SQLite Storage

    Admin->>API: POST /users (name, email, role="student")
    API->>AuthSvc: create_user(data)
    AuthSvc->>AuthSvc: Generate Salt & Temp Password Hash
    AuthSvc->>DB: INSERT INTO users (status='Pending', active=0)
    AuthSvc->>AuthSvc: Generate 32-Byte Token (secrets.token_urlsafe)
    AuthSvc->>DB: INSERT INTO activation_tokens (user_id, token, expires_at)
    AuthSvc->>Mail: send_activation_email(email, token)
    Mail-->>Student: Email with Link /activate?token=<token>

    Note over Student,API: Account Activation Phase
    Student->>API: POST /auth/activate (token, new_password)
    API->>AuthSvc: activate_account(token, new_password)
    AuthSvc->>DB: SELECT * FROM activation_tokens WHERE token = ?
    AuthSvc->>AuthSvc: Hash new_password with PBKDF2 (100,000 iter)
    AuthSvc->>DB: UPDATE users SET status='Active', active=1, password_hash=?, salt=?
    AuthSvc->>DB: DELETE FROM activation_tokens WHERE token = ?
    API-->>Student: 200 OK (Account Activated)

    Note over Student,API: Authentication & Invalidation
    Student->>API: POST /auth/login (email, new_password)
    API->>AuthSvc: authenticate() & create_session()
    AuthSvc->>DB: INSERT INTO user_sessions (session_token=SHA256(raw_token), user_id)
    API-->>Student: 200 OK (access_token = raw_token)

    opt Administrative Deactivation
        Admin->>API: PUT /users/:id/deactivate
        API->>AuthSvc: deactivate_user(user_id)
        AuthSvc->>DB: UPDATE users SET status='Inactive', active=0
        AuthSvc->>DB: DELETE FROM user_sessions WHERE user_id = ?
        Note over Student,API: All Active Sessions Immediately Revoked
    end
```

---

5. Official Announcement Publishing & Access Workflow

This workflow traces circular publishing, RBAC enforcement, dynamic administrator contact retrieval, and mandatory student acknowledgements.

```
sequenceDiagram
    autonumber
    actor Staff as Admin / App Manager / Warden
    actor Student as Student
    participant Page as AnnouncementPage.jsx
    participant API as announcements.py
    participant DB as SQLite Storage

    Note over Staff: Staff Notice Publishing
    Staff->>Page: Fills Title, Body, Category, Target, ReqAck
    Staff->>API: POST /announcements (JSON + Bearer Token)
    API->>API: require_roles("administrator", "app_manager", "location_in_charge")
    API->>DB: INSERT INTO announcements (...)
    API->>DB: INSERT INTO announcement_targets (...)
    API-->>Staff: 201 Created

    Note over Student: Student Access & Guarding
    Student->>Page: Opens /announcements
    Page->>Page: canCreate = false -> Hides "Compose Notice" button & modal
    Page->>API: GET /announcements/admin-contact
    API->>DB: Query active Administrator in users table
    DB-->>API: Returns USR_ADMIN (gowthamejjurothu@gmail.com)
    API-->>Page: 200 OK { available: true, email: "gowthamejjurothu@gmail.com" }
    Page->>Student: Renders Admin Contact mailto link (Zero Hardcoded Emails)

    opt Mandatory Acknowledgement
        Student->>Page: Clicks "Acknowledge Notice"
        Page->>API: PUT /announcements/:id/acknowledge
        API->>DB: INSERT INTO announcement_acknowledgements (announcement_id, user_id)
        API-->>Student: 200 OK (Acknowledged)
    end
```

CAMPUS PULSE KNOWLEDGE SUITE

Part IV: Relational Database & Schema Specification

Source: `CAMPUS_PULSE_DATABASE_REFERENCE.md`

Campus Pulse — Database Architecture, Schema Reference & Relational Model

Document Type: Complete Relational Schema Specification & Catalog Reference
Repository: Campus Pulse (`CampusPulse_co`)
Persistence Engine: SQLite 3 (WAL Mode)
Database Location: `Backend/storage/campus_pulse.db`

---

1. Engine Configuration & Relational Overview

The runtime database for Campus Pulse is SQLite 3, located at `Backend/storage/campus_pulse.db`. Inspection of the system catalog (`sqlite_master`) reveals **196 total tables**.

```
erDiagram
    users ||--o{ user_sessions : "has sessions"
    users ||--o| student_profiles : "profile"
    users ||--o| staff_profiles : "profile"
    users ||--o{ complaints : "reported by"
    users ||--o{ coordinator_assignments : "assigned as"
    users ||--o{ notifications : "receives"
    users ||--o{ audit_logs : "actor in"

    locations ||--o{ complaints : "location of"
    locations ||--o{ coordinator_assignments : "scoped to"
    locations ||--o{ staff_profiles : "facility"

    departments ||--o{ complaints : "department of"
    departments ||--o{ coordinator_assignments : "scoped to"
    departments ||--o{ department_issues : "defines"

    complaints ||--o{ comments : "has discussion"
    complaints ||--o{ attachments : "evidence"
    complaints ||--o{ affected_students : "upvoted by"

    announcements ||--o{ announcement_reads : "read tracking"
    announcements ||--o{ announcement_acknowledgements : "student ack"
    announcements ||--o{ announcement_targets : "target scope"
```

PRAGMA Settings & Execution Environment

On every connection created in `Backend/services/auth_service.py` and `Backend/services/complaint_service.py`, the following PRAGMAs are executed:

```
PRAGMA journal_mode = WAL;          -- Write-Ahead Logging allows unlimited concurrent readers
PRAGMA foreign_keys = ON;           -- Enforces relational foreign key cascades and integrity
PRAGMA synchronous = NORMAL;        -- Ensures durability on checkpoints with optimal write speed
PRAGMA busy_timeout = 5000;         -- 5000ms retry window before throwing 'database is locked'
```

---

2. Active Core Tables vs. Scaffolded Enterprise Schema

A critical discovery of this codebase audit is distinguishing between **Active Operational Tables** and **Scaffolded Enterprise Schema Tables**:

1. **Active Operational Tables (~35 Tables):**
2. Directly wired to active FastAPI routers, service classes, UI forms, and automated tests.
3. Core tables include: `users` , `student_profiles` , `staff_profiles` , `locations` , `departments` , `department_issues` , `coordinator_assignments` , `complaints` , `comments` , `attachments` , `affected_students` , `user_sessions` , `login_attempts` , `login_history` , `activation_tokens` , `password_resets` , `notifications` , `announcements` , `announcement_reads` , `announcement_acknowledgements` , `announcement_targets` , `lost_found_items` , `lost_found_images` , `feedback` , `chat_conversations` , `chat_messages` , `audit_logs` , `backups` , `system_settings` , `institutions` .
4. **Scaffolded Enterprise Schema Placeholders (~160 Tables):**
5. Tables such as `visitors` , `visitor_documents` , `visitor_gate_pass` , `work_orders` , `work_order_parts` , `asset_assignments` , `asset_disposals` , `ai_predictions` , `ai_models` , `marketplace_packages` , `failover_targets` , `offline_sync_queue` , `privacy_requests` , `virus_scan_logs` .
6. **Verification Finding:** These tables were generated by enterprise database migrations and schema scaffolding scripts (`Backend/scripts/`). They contain empty schemas or sample definitions, but **no active API routers in `Backend/api/routers/` or pages in `Frontend/src/pages/` currently query or update them**.
7. **Guidance:** They do not impair performance or stability, but developers should not assume features like visitor check-in or AI asset maintenance exist in the user interface.

---

3. Core Relational Tables Specification

3.1 `users`

Central user identity and authentication record.

```
CREATE TABLE users (
    user_id TEXT PRIMARY KEY,
    email TEXT UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    salt TEXT NOT NULL,
    role TEXT NOT NULL,                -- Enum: super_admin, administrator, app_manager, location_in_charge, coordinator, student
    name TEXT NOT NULL,
    status TEXT DEFAULT 'Active',      -- Enum: Active, Inactive, Pending, Suspended, Archived
    active INTEGER DEFAULT 1,          -- 1 = Active, 0 = Inactive
    is_deleted INTEGER DEFAULT 0,      -- Soft delete flag
    is_temp_password INTEGER DEFAULT 0,-- 1 requires password change
    institution_id TEXT DEFAULT NULL,
    phone_number TEXT DEFAULT NULL,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_role ON users(role);
CREATE INDEX idx_users_status ON users(status);
```

3.2 `student_profiles`

Academic details and hostel assignment scoping.

```
CREATE TABLE student_profiles (
    student_id TEXT PRIMARY KEY REFERENCES users(user_id) ON DELETE CASCADE,
    roll_number TEXT,
    program_id TEXT,
    year_of_study INTEGER,
    hostel_location_id TEXT REFERENCES locations(location_id),
    room_number TEXT,
    emergency_contact TEXT
);
CREATE INDEX idx_student_hostel ON student_profiles(hostel_location_id);
```

3.3 `locations`

Physical campus facilities.

```
CREATE TABLE locations (
    location_id TEXT PRIMARY KEY,       -- e.g., LOC_HOSTEL_1, LOC_ACADEMIC_1
    name TEXT NOT NULL,
    location_type TEXT NOT NULL,        -- Enum: Hostel, Academic, Mess, SAC, Sports, Other
    app_manager_id TEXT REFERENCES users(user_id),
    warden_id TEXT REFERENCES users(user_id),
    status TEXT DEFAULT 'Active',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_locations_type ON locations(location_type);
CREATE INDEX idx_locations_manager ON locations(app_manager_id);
CREATE INDEX idx_locations_warden ON locations(warden_id);
```

3.4 `departments`

Maintenance and operational service departments.

```
CREATE TABLE departments (
    department_id TEXT PRIMARY KEY,     -- e.g., DEPT_PLUMBING, DEPT_ELECTRICAL
    name TEXT NOT NULL,
    code TEXT,
    head_of_dept TEXT,
    status TEXT DEFAULT 'Active',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);
```

3.5 `coordinator_assignments`

Maps departmental coordinators to specific facilities.

```
CREATE TABLE coordinator_assignments (
    assignment_id TEXT PRIMARY KEY,
    coordinator_id TEXT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    location_id TEXT NOT NULL REFERENCES locations(location_id) ON DELETE CASCADE,
    department_id TEXT NOT NULL REFERENCES departments(department_id) ON DELETE CASCADE,
    is_primary INTEGER DEFAULT 1,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_coord_asgn_loc_dept ON coordinator_assignments(location_id, department_id);
CREATE INDEX idx_coord_asgn_coord ON coordinator_assignments(coordinator_id);
```

3.6 `complaints`

Grievance record and state machine status.

```
CREATE TABLE complaints (
    complaint_id TEXT PRIMARY KEY,      -- e.g., CMP-XXXXXX
    title TEXT NOT NULL,
    description TEXT NOT NULL,
    category TEXT,
    location_id TEXT NOT NULL REFERENCES locations(location_id),
    department_id TEXT NOT NULL REFERENCES departments(department_id),
    created_by TEXT NOT NULL REFERENCES users(user_id),
    assigned_to TEXT REFERENCES users(user_id),
    status TEXT DEFAULT 'Submitted',    -- Submitted, Assigned, In Progress, Resolved, Waiting for Verification, Closed, Closed (Auto Closed), Reopened, Rejec
    priority TEXT DEFAULT 'Normal',     -- Low, Normal, High, Critical
    sla_deadline TEXT,
    is_escalated INTEGER DEFAULT 0,
    reopen_count INTEGER DEFAULT 0,
    resolution_notes TEXT,
    resolved_at TEXT,
    closed_at TEXT,
    verification_due_at TEXT,
    visibility_type TEXT DEFAULT 'Private', -- Private, Public, Community
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_complaints_created_by ON complaints(created_by);
CREATE INDEX idx_complaints_assigned_to ON complaints(assigned_to);
CREATE INDEX idx_complaints_status ON complaints(status);
CREATE INDEX idx_complaints_loc_dept ON complaints(location_id, department_id);
CREATE INDEX idx_complaints_escalated ON complaints(is_escalated);
```

3.7 `user_sessions`

Active opaque session token hash table.

```
CREATE TABLE user_sessions (
    session_token TEXT PRIMARY KEY,     -- SHA-256 hex digest of the raw bearer token
    user_id TEXT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    created_at REAL NOT NULL,           -- Epoch timestamp
    expires_at REAL NOT NULL,           -- Epoch timestamp
    last_seen REAL NOT NULL             -- Epoch timestamp
);
CREATE INDEX idx_sessions_user_id ON user_sessions(user_id);
```

3.8 `announcements`

Official circulars, circular targets, and acknowledgements.

```
CREATE TABLE announcements (
    announcement_id TEXT PRIMARY KEY,
    title TEXT NOT NULL,
    content TEXT NOT NULL,
    summary TEXT,
    category TEXT DEFAULT 'General',
    priority TEXT DEFAULT 'Normal',
    is_pinned INTEGER DEFAULT 0,
    requires_ack INTEGER DEFAULT 0,
    status TEXT DEFAULT 'Published',
    created_by TEXT NOT NULL REFERENCES users(user_id),
    published_at TEXT,
    expires_at TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE announcement_acknowledgements (
    ack_id TEXT PRIMARY KEY,
    announcement_id TEXT NOT NULL REFERENCES announcements(announcement_id) ON DELETE CASCADE,
    user_id TEXT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    remarks TEXT,
    acknowledged_at TEXT DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(announcement_id, user_id)
);
```

3.9 `audit_logs`

Immutable administrative audit trail.

```
CREATE TABLE audit_logs (
    log_id INTEGER PRIMARY KEY AUTOINCREMENT,
    action TEXT NOT NULL,
    entity_type TEXT NOT NULL,
    entity_id TEXT,
    actor_id TEXT NOT NULL,
    details TEXT,                       -- JSON string of parameters or diffs
    ip_address TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_audit_actor ON audit_logs(actor_id);
CREATE INDEX idx_audit_action ON audit_logs(action);
CREATE INDEX idx_audit_created ON audit_logs(created_at);
```

---

4. Concurrency, Serialization & Data-Integrity Risks

1. **SQLite Database-Level Write Lock:**
2. SQLite enforces single-writer serialization. If concurrent write transactions are attempted simultaneously (e.g., bulk complaint import while students are submitting issues), secondary transactions will wait up to `busy_timeout = 5000` ms.
3. If a transaction takes longer than 5 seconds, an `OperationalError: database is locked` occurs.
4. **Protection:** All service write methods use short-lived context managers that commit immediately.
5. **JSON Serialized Fields:**
6. In `audit_logs.details`, state diffs are serialized as JSON strings.
7. In `complaints.verification_reports` and `complaints.history` (when using object serialization), history logs are stored as serialized lists.
8. **Integrity Rule:** Updates to these fields must deserialize, append, and reserialize safely within a transaction.
9. **Database Backup Architecture:**
10. Backups are performed via SQLite's Online Backup API (`sqlite3.Connection.backup()`). This creates a consistent point-in-time snapshot of the database file on disk without corrupting active WAL readers.

CAMPUS PULSE KNOWLEDGE SUITE

Part V: Complete REST API Endpoint Reference (277 Routes)

Source: `CAMPUS_PULSE_API_REFERENCE.md`

Campus Pulse — REST API Reference, Routing Registry & Security Directory

Document Type: Complete REST API Specification & Endpoint Directory
Repository: Campus Pulse (`CampusPulse_co`)
Total Registered Routes: 277 Endpoints across 24 Routers
Base URL: `http://127.0.0.1:8000` (Direct) or `/api` (Reverse Proxy)

---

1. API Architecture & Standards

- **Protocol:** HTTP/1.1 with TLS (Production); WebSockets on `/ws/{user_id}`
- **Authentication:** `Authorization: Bearer <token>` header containing opaque 32-byte session token
- **Response Format:** `application/json` (REST endpoints); `multipart/form-data` (file uploads)
- **Error Standard:** Standard RFC 7807 JSON error body: `json { "detail": "Error message description or validation array" }`
- **Global Middleware:** Rate limiter, CORS header injector, security headers (`nosniff` , `DENY` , conditional `HSTS`), request timing logger.

---

2. API Endpoints Catalog by Module

Module 1: Authentication & Account Management (`/auth`)

Source Router: `Backend/api/routers/auth.py`
Covered Tests: `Backend/tests/test_workflow1_auth.py` , `Backend/tests/smoke_test.py`

Method	Path	Purpose	Required Roles	Auth	Request Body / Params	Database Effects	Status Codes
`POST`	`/auth/login`	Authenticates user & issues session token	Public	No	`LoginRequest(username, password)`	Inserts `login_attempts` , inserts `user_sessions`	200, 401, 403, 429
`POST`	`/auth/logout`	Revokes active session token	Any	Yes	Bearer Token	Deletes token from `user_sessions`	200, 401
`GET`	`/auth/me`	Fetches authenticated caller profile	Any	Yes	None	Reads `users` & profile table	200, 401
`POST`	`/auth/change-password`	Updates password & revokes other sessions	Any	Yes	`PasswordChangeRequest(old_password, new_password)`	Updates `users.password_hash` , revokes `user_sessions`	200, 400, 401
`POST`	`/auth/forgot-password`	Sends password reset email link	Public	No	`ForgotPasswordRequest(email)`	Inserts `password_resets` , sends email	200, 404
`POST`	`/auth/reset-password`	Resets password via email token	Public	No	`ResetPasswordRequest(token, new_password)`	Updates `users.password_hash` , deletes reset token	200, 400
`POST`	`/auth/activate`	Activates account via activation token	Public	No	`ActivateAccountRequest(token, password)`	Updates `users.status='Active'` , deletes token	200, 400

Module 2: User & Roster Management (`/users`)

Source Router: `Backend/api/routers/users.py`
Covered Tests: `Backend/tests/test_workflow2_user_management.py`

Method	Path	Purpose	Required Roles	Auth	Request Body / Params	Database Effects	Status Codes
`GET`	`/users`	Lists users with role/status filters	Admin, App Manager	Yes	`?role=&status=&q=&limit=&skip=`	Reads `users`	200, 401, 403
`POST`	`/users`	Creates single user account	Admin, App Manager	Yes	`UserCreateRequest(...)`	Inserts `users` , `activation_tokens`	201, 400, 403

Method	Path	Purpose	Required Roles	Auth	Request Body / Params	Database Effects	Status Codes
`GET`	`/users/{user_id}`	Retrieves full user profile	Admin, Manager, Self	Yes	Path: `user_id`	Reads `users`, `student_profiles`	200, 404
`PUT`	`/users/{user_id}`	Updates user information	Admin, Manager	Yes	`UserUpdateRequest(...)`	Updates `users`, logs to `audit_logs`	200, 400, 403
`PUT`	`/users/{user_id}/deactivate`	Deactivates user & revokes sessions	Admin, Manager	Yes	Path: `user_id`	Updates `users.status='Inactive'`, clears `user_sessions`	200, 403, 404
`PUT`	`/users/{user_id}/activate`	Reactivates inactive account	Admin, Manager	Yes	Path: `user_id`	Updates `users.status='Active'`, `users.active=1`	200, 403, 404
`DELETE`	`/users/{user_id}`	Soft-deletes user	Admin	Yes	Path: `user_id`	Sets `is_deleted=1` (or cascades if safe)	200, 403, 404
`POST`	`/users/import`	Bulk imports students from CSV/XLSX	Admin, Manager	Yes	`UploadFile` (multipart)	Batch inserts `users`, `student_profiles`	200, 400, 403

Module 3: Facility & Location Management (`/locations`)

Source Router: `Backend/api/routers/locations.py`
Covered Tests: `Backend/tests/test_role_location_assignment.py`

Method	Path	Purpose	Required Roles	Auth	Request Body / Params	Database Effects	Status Codes
`GET`	`/locations`	Lists campus locations & buildings	Any	Yes	`?type=&status=`	Reads `locations`	200, 401
`POST`	`/locations`	Creates new campus location	Admin, Manager	Yes	`LocationCreateRequest(...)`	Inserts `locations`, logs `audit_logs`	201, 400, 403
`GET`	`/locations/{id}`	Gets location details & wardens	Any	Yes	Path: `id`	Reads `locations`	200, 404
`PUT`	`/locations/{id}`	Updates location assignments	Admin, Manager	Yes	`LocationUpdateRequest(...)`	Updates `locations.app_manager_id`, `warden_id`	200, 400, 403
`DELETE`	`/locations/{id}`	Deactivates or removes location	Admin	Yes	Path: `id`	Updates `locations.status='Inactive'`	200, 400, 403

Module 4: Department Management (`/departments`)

Source Router: `Backend/api/routers/departments.py`

Method	Path	Purpose	Required Roles	Auth	Request Body / Params	Database Effects	Status Codes
`GET`	`/departments`	Lists all service departments	Any	Yes	`?status=`	Reads `departments`	200, 401
`POST`	`/departments`	Creates service department	Admin, Manager	Yes	`DepartmentCreateRequest(...)`	Inserts `departments`	201, 400, 403
`GET`	`/departments/{id}`	Gets department details	Any	Yes	Path: `id`	Reads `departments`	200, 404
`PUT`	`/departments/{id}`	Updates department config	Admin, Manager	Yes	`DepartmentUpdateRequest(...)`	Updates `departments`	200, 400, 403
`DELETE`	`/departments/{id}`	Deactivates department	Admin	Yes	Path: `id`	Sets `departments.status='Inactive'`	200, 400, 403

Module 5: Complaint Operations (`/complaints`)

Source Router: `Backend/api/routers/complaints.py`
Covered Tests: `Backend/tests/test_workflow4_complaint_management.py`

Method	Path	Purpose	Required Roles	Auth	Request Body / Params	Database Effects	Status Codes
`GET`	`/complaints`	Lists complaints scoped to caller role	Any	Yes	`?status=&priority=&location_id=&q=`	Reads `complaints` (filtered by `_visible_complaints`)	200, 401, 403
`POST`	`/complaints`	Lodges new student complaint	Student, Warden	Yes	`ComplaintCreateRequest(...)`	Inserts `complaints`, `notifications`	201, 400, 403
`GET`	`/complaints/{id}`	Gets full complaint detail & comments	Author, Staff	Yes	Path: `id`	Verified by `_ensure_visible()`	200, 403, 404
`PUT`	`/complaints/{id}/status`	Updates complaint status	Staff, Admin	Yes	`StatusUpdateRequest(status)`	Updates `complaints.status`	200, 400, 403
`PUT`	`/complaints/{id}/assign`	Reassigns complaint to coordinator	Admin, Manager, Warden	Yes	`AssignRequest(assigned_to)`	Updates `complaints.assigned_to`	200, 400, 403
`PUT`	`/complaints/{id}/resolve`	Marks complaint resolved with proof	Coordinator, Staff	Yes	`ResolveRequest(notes, proof_url)`	Updates status to `'Waiting for Verification'`	200, 400, 403
`PUT`	`/complaints/{id}/verify`	Approves resolution & closes issue	Complainant Student	Yes	`VerifyRequest(rating, comments)`	Updates status to `'Closed'`	200, 400, 403
`POST`	`/complaints/{id}/reopen`	Reopens unsatisfactory resolution	Complainant Student	Yes	`ReopenRequest(reason)`	Updates status to `'Reopened'`, increments count	200, 400, 403
`POST`	`/complaints/{id}/comments`	Appends discussion comment	Author, Staff	Yes	`CommentCreateRequest(content)`	Inserts `comments`	201, 400, 403
`POST`	`/complaints/{id}/upvote`	Upvotes public complaint	Student	Yes	Path: `id`	Inserts `affected_students`	200, 400

Module 6: Campus Announcements & Notices (`/announcements`)

Source Router: `Backend/api/routers/announcements.py`
Covered Tests: `Backend/tests/test_workflow8_announcements.py`

Method	Path	Purpose	Required Roles	Auth	Request Body / Params	Database Effects	Status Codes
`GET`	`/announcements`	Fetches active campus circulars	Any	Yes	`?category=&priority=&q=`	Reads `announcements`	200, 401
`POST`	`/announcements`	Composes & broadcasts notice	**Admin, Manager, Warden**	Yes	`AnnouncementCreateRequest(...)`	Inserts `announcements`, `announcement_targets`	**201, 400, 403**
`GET`	`/announcements/admin-contact`	Dynamically gets admin contact email	Any	Yes	None	Reads active `administrator` in `users`	200, 401
`GET`	`/announcements/{id}`	Retrieves circular detail & metrics	Any	Yes	Path: `id`	Reads `announcements`	200, 404
`PUT`	`/announcements/{id}`	Updates announcement text	Creator, Admin	Yes	`AnnouncementUpdateRequest(...)`	Updates `announcements`	200, 400, 403
`DELETE`	`/announcements/{id}`	Deletes notice	Admin, Manager	Yes	Path: `id`	Deletes or archives notice	200, 403, 404
`POST`	`/announcements/{id}/read`	Records user read receipt	Any	Yes	Path: `id`	Inserts `announcement_reads`	200, 404
`PUT`	`/announcements/{id}/acknowledge`	Submits mandatory student acknowledgement	Student	Yes	`AcknowledgeRequest(remarks)`	Inserts `announcement_acknowledgements`	200, 400

Module 7: Notifications & WebSockets (`/notifications`, `/ws`)

Source Router: `Backend/api/routers/notifications.py`

Method	Path	Purpose	Required Roles	Auth	Request Body / Params	Database Effects	Status Codes
`GET`	`/notifications`	Lists paginated notifications for caller	Any	Yes	`?limit=&offset=`	Reads `notifications`	200, 401
`PUT`	`/notifications/{id}/read`	Marks notification as read	Caller	Yes	Path: `id`	Updates `notifications.is_read = 1`	200, 404
`PUT`	`/notifications/read-all`	Marks all notifications as read	Caller	Yes	None	Updates all user rows in `notifications`	200, 401
`DELETE`	`/notifications/{id}`	Dismisses notification	Caller	Yes	Path: `id`	Deletes from `notifications`	200, 404
`WS`	`/ws/{user_id}`	WebSocket stream for live alerts	Caller	Token Query	WS Connection	Registers socket in `WebSocketManager`	101, 403

Module 8: Governance & System Maintenance (`/audit` , `/backups` , `/settings`)

Method	Path	Purpose	Required Roles	Auth	Request Body / Params	Database Effects	Status Codes
`GET`	`/audit`	Lists immutable audit log entries	Admin, Manager	Yes	`?action=&entity_type=&limit=`	Reads `audit_logs`	200, 403
`GET`	`/backups`	Lists database backups on disk	Admin, Manager	Yes	None	Reads `backups` table & filesystem	200, 403
`POST`	`/backups`	Triggers immediate SQLite online backup	Admin, Manager	Yes	None	Creates backup copy on disk	201, 500
`POST`	`/backups/{id}/restore`	Restores database snapshot	Admin	Yes	Path: `id`	Replaces active `campus_pulse.db`	200, 403, 500
`GET`	`/settings`	Retrieves system configuration keys	Admin	Yes	None	Reads `system_settings`	200, 403
`PUT`	`/settings`	Updates institutional configurations	Admin	Yes	`SettingsUpdateRequest(...)`	Updates `system_settings`	200, 403

Module 9: Multi-Tenant Platform Operations (`/platform` , `/superadmin`)

Source Router: `Backend/api/routers/platform.py`

Method	Path	Purpose	Required Roles	Auth	Request Body / Params	Database Effects	Status Codes
`GET`	`/superadmin/institutions`	Lists all campus tenant institutions	Super Admin	Yes	None	Reads `institutions` table	200, 403
`POST`	`/superadmin/institutions`	Registers new university institution	Super Admin	Yes	`InstitutionCreateRequest(...)`	Inserts `institutions`	201, 400, 403
`GET`	`/superadmin/health`	Comprehensive cluster health status	Super Admin	Yes	None	Checks disk, database WAL, schedulers	200, 403
`GET`	`/superadmin/support`	Global cross-institution support tickets	Super Admin	Yes	None	Reads platform support queue	200, 403

CAMPUS PULSE KNOWLEDGE SUITE

Part VI: Technical Issues, Architectural Gaps & Risk Register

Source: `CAMPUS_PULSE_ISSUES_AND_GAPS.md`

Campus Pulse — Issues, Gaps, and Technical Risks Register

Document ID: `CP-DOC-005`
Classification: Evidence-Based Source Code Investigation
Status: Complete
Date: October 2026

---

1. Overview and Methodology

This document details all confirmed technical issues, architectural limitations, security risks, documentation contradictions, and dead code identified during the comprehensive source code audit of the **Campus Pulse** repository (`CampusPulse_co`).

Each issue is assigned a unique tracking identifier, categorized, evaluated for severity, and mapped directly to source code references with reproduction mechanisms and remediation paths.

---

2. Issues Summary Matrix

Issue ID	Severity	Category	Title	Affected File(s)
ISSUE-001	High	Concurrency / DB	SQLite Single-Writer Lock Bottleneck	`Backend/database/session.py` , `Backend/storage/campus_pulse.db`
ISSUE-002	High	Architecture / Concurrency	In-Process Thread Scheduler Multi-Worker Duplication	`Backend/services/scheduler_service.py` , `Backend/main.py`
ISSUE-003	Medium	Scalability / Realtime	In-Memory WebSocket Manager Lacks Cross-Worker Bus	`Backend/api/routes/websockets.py` , `Backend/services/websocket_service.py`
ISSUE-004	Medium	Performance	Synchronous SMTP Dispatch Blocking HTTP Request Threads	`Backend/services/email_service.py` , `Backend/api/routes/auth.py`
ISSUE-005	Medium	Schema Governance	160+ Enterprise Placeholder Tables (Schema Drift)	`Backend/storage/campus_pulse.db` , `Backend/models/`
ISSUE-006	Low	Storage / Deployment	Local Disk File Attachment Storage in Stateless Deployments	`Backend/api/routes/attachments.py` , `Backend/storage/uploads/`
ISSUE-007	High	Doc Contradiction	Documented SMS OTP Flow Non-Existent in Source Code	`Backend/api/routes/auth.py` , Legacy Documentation
ISSUE-008	Medium	Doc Contradiction	Documented AI Predictive Dispatch Is Scaffolded Only	`Backend/storage/campus_pulse.db` (`ai_predictions`)
ISSUE-009	Medium	Doc Contradiction	Super Admin Operational Complaint Viewing Blocked	`Backend/api/routes/complaints.py` (`_visible_complaints`)
ISSUE-010	Low	Workflow Edge Case	Reopen Count Limit Reached Lacks Escalation Alert	`Backend/api/routes/complaints.py` , `Backend/models/complaint.py`

---

3. Detailed Issue Specifications

ISSUE-001: SQLite Single-Writer Lock Bottleneck

- **Category:** Concurrency / Database Architecture
- **Severity:** High
- **Affected Files:**
- `Backend/database/session.py`
- `Backend/storage/campus_pulse.db`
- **Description & Evidence:** The production backend uses SQLite 3 configured with `PRAGMA journal_mode=WAL` and `PRAGMA busy_timeout=5000` . While Write-Ahead Logging allows multiple concurrent readers alongside one active writer, write transactions are strictly serialized at the operating system file lock level. Under burst conditions (e.g., hundreds of students filing complaints following a campus-wide outage), concurrent `POST /complaints` , attachment uploads, and background SLA status updates can hold the write lock longer than 5,000ms, triggering `sqlite3.OperationalError: database is locked` .
- **Reproduction:**
- Launch 50 concurrent asynchronous threads submitting complaints via `POST /api/complaints` .

- Background scheduler simultaneously executes batch updates on `complaints` for SLA escalation.
- Tail requests log `sqlite3.OperationalError` with HTTP 500 status.
- **Impact:** Request timeouts and transaction failures during high-traffic campus incidents.
- **Remediation:** For enterprise deployment beyond pilot scale, migrate the SQLAlchemy engine connection string to PostgreSQL (`postgresql+psycopg2://...`).

ISSUE-002: In-Process Thread Scheduler Multi-Worker Duplication

- **Category:** Background Jobs / Architecture
- **Severity:** High
- **Affected Files:**
- `Backend/services/scheduler_service.py`
- `Backend/main.py`
- **Description & Evidence:** In `Backend/main.py` , the SLA checker and auto-close background runner is started in an in-process daemon thread (`scheduler_service.start_scheduler()`) on FastAPI lifespan startup. If Uvicorn or Gunicorn is launched with multiple worker processes (e.g. `uvicorn main:app --workers 4`), each worker process spawns an independent scheduler thread. Every 300 seconds, 4 distinct threads query the same SQLite database for overdue complaints and trigger duplicate notification dispatches and audit log records.
- **Reproduction:**
- Start Uvicorn with `--workers 2` .
- Seed an overdue complaint past its 48-hour SLA deadline.
- Observe notifications table receiving duplicate escalation alerts simultaneously.
- **Impact:** Duplicate emails, notifications, and unnecessary database write lock contention.
- **Remediation:** Move periodic background tasks to a dedicated worker process (Celery or APScheduler running as a standalone single-instance service) or introduce distributed database lock leasing.

ISSUE-003: In-Memory WebSocket Manager Lacks Cross-Worker Bus

- **Category:** Real-Time Messaging / Scalability
- **Severity:** Medium
- **Affected Files:**
- `Backend/api/routes/websockets.py`
- `Backend/services/websocket_service.py`
- **Description & Evidence:** The `ConnectionManager` class maintains WebSocket connections inside a Python in-memory dictionary `self.active_connections: Dict[int, List[WebSocket]]` . When a backend route (e.g. complaint status change) invokes `manager.send_personal_message(user_id, data)` , it can only broadcast to connections managed by the current Python process. In a multi-worker or multi-container environment, if Student 1 is connected to Worker A and an update is processed on Worker B, Student 1 will not receive the real-time notification push.
- **Reproduction:**
- Connect a WebSocket client to Worker A.
- Issue a REST API status update that hits Worker B.
- The WebSocket client on Worker A receives no frame until page refresh.
- **Impact:** Inconsistent real-time UI updates in production multi-process clusters.
- **Remediation:** Introduce a Redis pub/sub backplane or ASGI channel layer (`broadcaster` library) to synchronize broadcast events across all worker processes.

ISSUE-004: Synchronous SMTP Dispatch Blocking HTTP Request Threads

- **Category:** Performance / Availability
- **Severity:** Medium
- **Affected Files:**
- `Backend/services/email_service.py`
- `Backend/api/routes/auth.py`
- **Description & Evidence:** `email_service.send_email()` establishes a synchronous `smtplib.SMTP` connection, executes TLS negotiation, and transmits messages directly within the HTTP request execution flow (or fast background task). If the external SMTP server has network latency, throttles requests, or times out, the user's HTTP request hangs for 10–30 seconds.
- **Reproduction:**
- Configure SMTP host to an unresponsive IP.
- Submit `POST /api/auth/register` or `POST /api/auth/request-password-reset` .
- The request blocks until socket timeout occurs before returning a response.
- **Impact:** Thread pool exhaustion, degraded UI responsiveness, and potential 504 Gateway Timeouts.
- **Remediation:** Offload email compilation and dispatch to an asynchronous background task queue (Celery, ARQ, or FastAPI `BackgroundTasks` with strict socket timeouts).

ISSUE-005: 160+ Enterprise Placeholder Tables (Schema Drift)

- **Category:** Database Schema Governance
- **Severity:** Medium

- **Affected Files:**
- `Backend/storage/campus_pulse.db`
- `Backend/models/`
- **Description & Evidence:** The SQLite catalog contains 196 tables. Detailed code analysis confirms that only ~35 tables are actively referenced by SQLAlchemy models, query builders, or API routes. The remaining ~160 tables (e.g. `visitor_logs`, `gate_passes`, `vendor_contracts`, `asset_inventories`, `ai_predictions`) are dormant artifacts from an expanded schema script. They possess foreign key constraints and indexes that consume schema catalog memory and mislead new developers.
- **Reproduction:** Run `SELECT tbl_name FROM sqlite_master WHERE type='table'` and cross-reference against `Backend/models/`.
- **Impact:** High cognitive overhead for maintenance developers, risk of accidental dependencies on dead tables, and inflated migration complexity.
- **Remediation:** Archive unused schema definitions into a separate enterprise roadmap migration file or drop unused placeholder tables from operational databases.

ISSUE-006: Local Disk File Attachment Storage in Stateless Deployments

- **Category:** Deployment / Cloud Scalability
- **Severity:** Low
- **Affected Files:**
- `Backend/api/routes/attachments.py`
- `Backend/storage/uploads/`
- **Description & Evidence:** Complaint attachment uploads are written directly to local disk directory `Backend/storage/uploads/{uuid}.{ext}`. In modern containerized deployments (Docker, Kubernetes, AWS ECS, Google Cloud Run) where container instances are ephemeral and autoscaled, attachments written on one instance cannot be read by another instance and are lost upon container recreation.
- **Reproduction:** Deploy two container replicas without shared persistent volumes; uploaded images on container A return HTTP 404 when requested through container B.
- **Impact:** Missing images and attachments in distributed or ephemeral cloud environments.
- **Remediation:** Implement an S3-compatible object storage provider abstraction (AWS S3, Cloudflare R2, MinIO) for attachment storage.

ISSUE-007: Documented SMS OTP Flow Non-Existent in Source Code

- **Category:** Documentation Contradiction
- **Severity:** High
- **Affected Files:**
- `Backend/api/routes/auth.py`
- Legacy documentation / design specs
- **Description & Evidence:** Several historical design summaries claim Campus Pulse uses "6-digit SMS OTP verification" for user activation and two-factor authentication. Inspection of the codebase reveals **no SMS provider integration** (no Twilio, AWS SNS, MSG91, or Fast2SMS libraries). Authentication and password reset rely strictly on 32-byte cryptographic URL tokens stored in `activation_tokens` and dispatched via email (`/activate?token=...`).
- **Impact:** Misleads security auditors and prospective institutional evaluators regarding the system's external communications infrastructure.
- **Remediation:** Update documentation to reflect reality (email token-based activation only) or specify SMS OTP as an unbuilt roadmap item.

ISSUE-008: Documented AI Predictive Dispatch Is Scaffolded Only

- **Category:** Documentation Contradiction / Feature Completeness
- **Severity:** Medium
- **Affected Files:**
- `Backend/storage/campus_pulse.db` (`ai_predictions` table)
- `Backend/api/routes/analytics.py`
- **Description & Evidence:** Marketing documents and legacy README files state that Campus Pulse incorporates an "AI-driven complaint categorization and predictive maintenance engine". While the database contains a table named `ai_predictions`, there are no machine learning libraries (`torch`, `scikit-learn`, `transformers`) imported in the backend requirements, nor is there any active inference pipeline during complaint creation. Categorization is driven strictly by user selection, keyword heuristics, and explicit department mapping.
- **Impact:** False perception of feature readiness.
- **Remediation:** Reclassify AI features in documentation as "Future Architecture Scaffold" rather than implemented capabilities.

ISSUE-009: Super Admin Operational Complaint Viewing Blocked

- **Category:** Documentation Contradiction / Permission Design
- **Severity:** Medium
- **Affected Files:**
- `Backend/api/routes/complaints.py` (`_visible_complaints`)
- **Description & Evidence:** General administrator documentation typically assumes `super_admin` has god-mode privileges across all data entities. In `complaints.py`, the helper function `_visible_complaints(current_user, db)` contains an explicit security boundary: `python if current_user.role == "super_admin": raise HTTPException( status_code=status.HTTP_403_FORBIDDEN, detail="Super Admins cannot access operational complaints." )` Super administrators are strictly limited to system-level configuration, tenant settings, and audit logs. Only `administrator`, `app_manager`, `location_in_charge`, `coordinator`, and `student` roles can query complaints.
- **Impact:** Administrators logging in under the `super_admin` credential expecting to view complaint queues will encounter HTTP 403 Forbidden errors if unaware of this architectural segregation.
- **Remediation:** Clearly document this role segregation in administrative runbooks.

ISSUE-010: Reopen Count Limit Reached Lacks Escalation Alert

- **Category:** Workflow Edge Case / Business Logic
- **Severity:** Low
- **Affected Files:**
- `Backend/api/routes/complaints.py`
- `Backend/models/complaint.py`
- **Description & Evidence:** When a student attempts to reopen a complaint, `complaint.reopened_count` is evaluated against a maximum threshold of 3. If `reopened_count >= 3`, the API returns HTTP 400 Bad Request with `"Maximum reopen limit reached"`. However, the system does not automatically flag the complaint as a "Disputed / Stalled" ticket nor does it trigger an automated alert to the App Manager or Administrator indicating that a student remains unsatisfied after 3 resolution attempts.
- **Impact:** Dissatisfied students reach a dead end without automated managerial oversight.
- **Remediation:** Introduce an automatic escalation workflow when `reopened_count >= 3` that notifies the App Manager and flags the complaint for administrative arbitration.

---

4. Accepted Limitations (By Design)

1. **SQLite as Single Storage Engine:**
2. Single-file database suitable for small-to-medium campus deployments (< 5,000 active concurrent users).
3. Low operational overhead, zero external database service configuration required for local pilot installations.
4. **Stateful Database Session Management:**
5. Stateful sessions stored in `user_sessions` enable instant single-session revocation, security auditing, and concurrent login limiting, accepting the minor trade-off of one database read per authenticated request.
6. **No Direct Student Announcement Creation:**
7. By policy and security enforcement, students cannot create campus-wide notices. Only authorized staff and administrators can publish official announcements.
8. **48-Hour Auto-Close Window:**
9. Auto-closing complaints 48 hours after reaching `Resolved` if the student takes no action is an intentional mechanism to prevent infinite queue backlogs.

CAMPUS PULSE KNOWLEDGE SUITE

Part VII: AI Handoff Prompt & Condensed Context

Source: `CAMPUS_PULSE_CONTEXT_FOR_CHATGPT.md`

Campus Pulse — AI System Prompt & Architectural Handoff Context

Target Audience: Incoming AI Assistants & Senior Engineers
Repository: Campus Pulse (`CampusPulse_co`)
Status: Ground-Truth Verified against Source Code (October 2026)
Instructions for the AI: You are inheriting the **Campus Pulse** codebase. Read this entire document before answering questions or proposing modifications. **Do not hallucinate features, do not suggest replacing the stack, and always inspect actual source code before answering.**

---

1. What Campus Pulse Is & Its Core Mission

Campus Pulse is a production-grade, multi-role campus operations and facility complaint management platform built for collegiate institutions. - **Problem Solved:** Decentralized, untracked university maintenance (hostels, academic blocks, mess halls, sports facilities) where student grievances are dropped, resolved without verification, or ignored without SLA escalation. - **Operating Philosophy:** - **Zero-Drop Routing:** Issues are automatically assigned to coordinators or escalated through facility wardens, app managers, and campus administrators. - **Student-First Closure:** Technicians cannot unilaterally close issues. Work completion initiates a mandatory **48-hour student verification window** before closure or auto-closure. - **Transparent Accountability:** Every status change, assignment, and administrative mutation is recorded in an immutable audit trail.

---

2. Actual Technology Stack

- **Backend:** Python 3 (running on Python 3.14.2) with **FastAPI** (`0.109.0+`) served by **Uvicorn** (`0.27.0+`).
- **Frontend: React 19** (`19.2.7`) with **Vite** (`8.1.3`), **React Router DOM v7** (`7.18.1`), and **Tailwind CSS v4** (`4.3.2`).
- **Database: SQLite 3** running with **Write-Ahead Logging (WAL)** (`Backend/storage/campus_pulse.db`).
- **HTTP Transport:** Axios (`1.18.1`) with centralized Bearer token interceptor (`Frontend/src/api/axios.js`).
- **Testing:** Pytest (`8.0+`) for backend API & workflows; Vitest (`3.0+`) for frontend React components.

---

3. The 6 User Roles & Strict Permissions

The application defines 6 user roles in `models/user.py` and enforces them via `api/security.py` :

```
Super Admin (Platform Level)
  └── Administrator (College Level)
        ├── App Manager (Multi-Facility Operations)
        │     └── Location In-Charge / Warden (Single Facility / Hostel)
        └── Coordinator (Departmental Technician / Handler)
              └── Student (Grievance Reporter / Resident)
```

1. `super_admin` **:** Multi-tenant platform manager (`/superadmin`). Strictly **blocked with 403 Forbidden** from accessing campus operational complaints (`complaints.py:96-100`).
2. `administrator` **:** Full institutional control (`/admin`). Manages users, locations, departments, backups, audit logs, and settings.
3. `app_manager` **:** Operational supervisor (`/manager`). Oversees assigned facilities (e.g., Hostel 1, Academic 1, Mess 1); reallocates complaint queues.
4. `location_in_charge` **(Warden):** Facility supervisor (`/location`). Manages issues in their specific building; posts hostel circulars.
5. `coordinator` **:** Service department technician (`/coordinator`). Resolves assigned complaints, uploads proof. **STRICTLY PROHIBITED FROM POSTING ANNOUNCEMENTS (403 Forbidden).**
6. `student` **:** Resident student (`/student`). Reports complaints (hostel issues restricted to assigned hostel); verifies or reopens resolutions; views notices. **STRICTLY PROHIBITED FROM POSTING ANNOUNCEMENTS (403 Forbidden).**

---

4. Key Business Rules & Workflows

4.1 Complaint Lifecycle State Machine

```
Submitted ──> Assigned ──> In Progress ──> Resolved ──> Waiting for Verification ──> Closed
    │             │                                          │
    └── Rejected ─┘                                          └── Reopened (Max 3x)
```

- **Resolution Proof:** A coordinator resolving a complaint must provide either an "After Repair" image attachment OR detailed resolution notes (`complaint_service.py:350-356`).
- **Verification Window:** Marking resolved sets status to `'Waiting for Verification'` with an auto-close due date (default: 48 hours / 5 days configured via `auto_close_days`).

- **Reopen Limit:** Students can reopen an issue up to 3 times (`max_reopen_count`). The 2nd reopen triggers an automated critical escalation alert to the Administrator.
- **Background Daemon:** Runs every 300 seconds (`services/scheduler_service.py`) to auto-close expired verification tickets and flag overdue SLA escalations.

4.2 Complaint Routing Engine

1. Matches `(location_id, department_id)` in `coordinator_assignments` .
2. Fallback: Routes to `locations.warden_id` (Facility Warden).
3. Fallback: Routes to `locations.app_manager_id` .
4. Fallback: Stays unassigned and alerts Administrator.

4.3 Official Announcements

- **Creation Entry Point:** Only rendered for Admin, App Manager, and Warden. Removed from Student UI.
- **Dynamic Admin Contact:** Non-staff viewing `/announcements` see the official Administrator email resolved dynamically via `GET /announcements/admin-contact` . **No hardcoded emails exist in the frontend.**

---

5. Authentication & Security Architecture

1. **Stateful Database Sessions (No Stateless JWTs):**
2. Issues 32-byte high-entropy token (`secrets.token_urlsafe(32)`).
3. Stores `SHA-256(token)` in SQLite table `user_sessions` .
4. Incoming requests check `Authorization: Bearer <token>` , hash it, and lookup `user_sessions` .
5. **Benefit:** Instant session invalidation on logout or administrative deactivation.
6. **Password Security:**
7. PBKDF2-HMAC-SHA256 with 100,000 iterations and 16-byte random salt (`utils/passwords.py`).
8. Constant-time verification (`secrets.compare_digest`).
9. **Account Activation Tokens (No SMS OTPs):**
10. Uses 32-byte cryptographic URL tokens in `activation_tokens` table.
11. Sent via email links; consumed upon initial password setup.
12. **Object-Level Authorization:**
13. Enforced by `_visible_complaints()` in `api/routers/complaints.py` . Students cannot view other students' private complaints by manipulating IDs.

---

6. What Is Implemented vs. Scaffolded / Incomplete

- **Fully Implemented & Verified:**
- Complete complaint lifecycle, multi-location assignment, and auto-close scheduler.
- Student, Coordinator, Warden, App Manager, Admin dashboards.
- Role-guarded announcements with dynamic admin contact.
- Lost & Found item reporting and photo attachments.
- In-app notification engine and WebSocket connections.
- Database backup and restore operations via SQLite Online Backup API.
- Audit logging of all administrative actions.
- **Scaffolded / Enterprise Schema Placeholders:**
- The SQLite database contains 196 tables. Approximately 130 tables (e.g., `visitors` , `work_orders` , `ai_predictions` , `marketplace_packages` , `failover_targets`) are scaffolded database schemas from enterprise planning. They are **not wired to active API endpoints or UI views**. Do not attempt to fix or delete them.

---

7. Known Architectural Limitations & Tech Debt

1. **SQLite Single-Writer Concurrency:**
2. WAL mode allows unlimited concurrent readers, but database writes serialize to a single writer. Adequate for single-campus use, but requires migration to PostgreSQL if scaling to tens of thousands of concurrent users.
3. **Synchronous SMTP Calls:**
4. `utils/email.py` performs synchronous network handshakes with port 587. Should be offloaded to FastAPI `BackgroundTasks` .
5. **In-Memory WebSocket Connections:**
6. `websocket_manager.py` maintains connections in a local Python dict. Multiple Gunicorn workers will require Redis Pub/Sub to broadcast across workers.

---

8. Critical Instructions for the AI Assistant

1. **DO NOT Refactor or Replace the Stack:** Do not suggest replacing FastAPI, React, SQLite, or Tailwind. The project is fully functional on these technologies.
2. **DO NOT Invent Roles or Modify Role Enums:** Roles are strictly `super_admin` , `administrator` , `app_manager` , `location_in_charge` , `coordinator` , and `student` .
3. **DO NOT Hardcode Credentials or Secrets:** Use environment variables from `Backend/.env` .
4. **ALWAYS Inspect Code First:** Before suggesting any change, inspect the actual file paths referenced in the detailed documentation.

---

9. Detailed Reference Documentation Index

For exhaustive technical specifics, refer to the files in `docs/project_knowledge/` : - **Complete Architecture & Workflows:** `CAMPUS_PULSE_MASTER_CONTEXT.md` - **All 277 API Endpoints:** `CAMPUS_PULSE_API_REFERENCE.md` - **Database Schema & Catalogs:** `CAMPUS_PULSE_DATABASE_REFERENCE.md` - **Mermaid Workflow Diagrams:** `CAMPUS_PULSE_WORKFLOWS.md` - **Defects & Technical Debt Register:** `CAMPUS_PULSE_ISSUES_AND_GAPS.md`